Phase-7 Structural Quant Engine

Audit Execution Instructions

Written August 27, 2026 — the procedure for running Step 3

Status: **working document.** Changes no rule, records no finding.

What this document is for.

The audit is run by hand, in a browser, possibly across several sessions days apart. This document exists so the procedure survives being put down and picked up again, without reconstructing it from a chat log. It also forms part of the audit record: under Item 6 (Traceability), how the auditor was asked bears directly on what the auditor found, so the exact wording used is preserved here rather than paraphrased.

Two runs, in a fixed order

Run 1 is a blind review. One model receives the source code and nothing else — no Constitution, no audit package. It is asked what is wrong with the code, with no knowledge of what the project claims about itself. Its value is narrow but real: it is the only run that can find a problem *the 44-rule register does not cover*, which is precisely the trigger that lifts the scope freeze.

Run 2 is the compliance audit — Step 3 proper. The intended primary auditor is Kimi K3 (Moonshot AI), per the Constitution's AUDITOR row. It receives the full material and evaluates the engine against the register, item by item, recording Compliant / Non-compliant / Unknown for each.

The order is not negotiable, and the reason is mechanical.

A model that has seen the Constitution cannot un-see it. Once it knows what the engine is supposed to do, it can no longer tell you what it would have noticed without knowing. Run 1 is therefore available exactly once, and only before Run 2. Running them out of order does not produce a worse blind review — it produces none at all.

1. Materials

All five files live in one folder. Byte sizes are given so a stale or truncated copy is caught before it reaches an auditor rather than after — this has already happened once, with an outdated copy of the Constitution missing the two rows Run 2 requires a finding on.

File	Bytes	Used in	What it is
`Phase7_Source_For_Audit.txt`	207,625	Runs 1 and 2	All 19 source files concatenated, each preceded by a delimiter naming its path. Roughly 56,000 tokens.
`Phase7_Engineering_Constitution_v1.0_RATIFIED_AUDITCOPY.pdf`	101,831	Runs A, B and C	The 44-rule register, 34 pages. The frozen audit copy — see the warning below. Deliberately not the same filename as the live document in docs/.
`Phase7_Step2a_Audit_Package.pdf`	21,386	Run 2 only	File manifest, evidence pointers, self-documented failure history, and two open questions. Assembled by Claude and therefore not neutral — Instruction 2 says so explicitly.
`Phase7_Step2a_Evidence.zip`	7,011	Run 2 only	Runtime log, scan summary, engine state file, one simulated trade record. None of these are in the public repository.
`Phase7_Step2a_Source.zip`	62,987	Neither	The same source as the .txt above, in original directory structure. A backup for the case where a plain-text upload fails.

Two Constitutions now exist, and they must not be swapped.

The copy in the audit folder is frozen at **101,831 bytes, 34 pages** — the exact document Runs A and B were graded against. The copy in `docs/` is the live one, and it grows: after Run B it gained an AUDITED row and became 104,090 bytes at 35 pages.

Run C must receive the frozen 101,831-byte copy. The AUDITED row states how many items came back Compliant and names which findings were rated Critical — handing that to the next auditor tells it what its predecessors concluded before it has read a line, which is the contamination the rules in Section 4 exist to prevent. All three runs are graded against one unchanging register; the live document records what the grading found.

The two files are therefore named differently on purpose. The audit folder holds `…_RATIFIED_AUDITCOPY.pdf` and nothing else called a Constitution; `docs/` holds `…_RATIFIED.pdf` and keeps growing. This is not tidiness. Both files briefly sat in the audit folder under the same name, and the wrong one was staged for upload within twenty minutes of this warning first being written — caught only because the byte count was read off a directory listing before sending. Check the size before every upload regardless; a distinct filename removes the way it went wrong the first time, not every way it could.

Where the runs happen

OpenRouter (openrouter.ai), which requires no subscription: credits are bought once, do not expire, and are drawn down per token. A full Run 2 costs roughly one US dollar even on the most expensive frontier models; Run 1 costs less. Cost is therefore not a constraint on how carefully this is done, and should not be treated as one.

2. Run 1 — Blind Review

Setup

Open a **new conversation**. Upload **only** `Phase7_Source_For_Audit.txt`. Nothing else from the folder may enter this conversation at any point — not the Constitution, not the audit package, not a summary of either, and not an explanation of what the engine is supposed to guarantee. Then paste the text below verbatim.

```
You are reviewing a Python codebase. I want your honest technical
assessment of what is wrong with it.

Context, kept deliberately minimal: this is a market-analysis engine for
a single cryptocurrency pair. It reads market data and produces analysis
output. It was written by one person over six days with heavy AI
assistance. That is all you need to know, and I am withholding further
context on purpose - I do not want you evaluating it against anyone
else's stated intentions, including mine.

Read the source and tell me what is actually wrong with it. Prioritise:

1. Anything that would produce silently incorrect output - wrong numbers
   that look plausible, rather than crashes.
2. Anything where the code could do something the reader of it would not
   expect. State plainly if you cannot determine from the code alone what
   the program is capable of doing.
3. Structural or architectural problems that will cause real trouble
   later, as opposed to style preferences.
4. Anything that looks like it was written to work in one specific case
   and would break in adjacent ones.

Rank findings by how much damage they could cause, not by how easy they
are to fix. Be specific: name the file and, where you can, the function.

Do not soften the assessment. Do not open with praise. If large parts of
it are fine, say so in one line and spend the response on what is not.

If you find nothing serious, say that plainly - do not manufacture
findings to seem thorough.
```

What a good result looks like

Expect noise. A reviewer with no context will raise generic points — missing tests, absent type hints, broad exception handling — that may be true but are not what this run is for. The signal to watch for is narrower and worth reading carefully: a **serious problem that none of the 44 items would have caught**. That is the finding that matters, because it is evidence the register itself has a hole, and under the scope freeze that is the specific condition that permits the register to change.

> *A second thing worth noting: if the reviewer says it cannot determine from the code alone whether the program can place trades, that is a real finding about the code's legibility, not a failure of the review. It was deliberately not told the answer.*

3. Run 2 — Compliance Audit (Step 3)

Setup

Open a **separate new conversation** — not a continuation of Run 1, and not a conversation that has previously discussed this project. Select Kimi K3 (Moonshot AI). Upload four files: the Constitution, the source, the audit package, and the evidence archive. Then paste the text on the next page verbatim.

Notes on running it

The full material is roughly 100,000 tokens of input. That fits comfortably in Kimi K3's context window, but if the model's replies begin referring vaguely to material it should be able to quote exactly, suspect truncation and check that all four files were actually received.

If the audit stalls partway — a long register is a long task — asking it to continue from a named item is legitimate. Asking it to summarise, shorten, or skip ahead is not: an item without a recorded finding is an item the scope freeze still rests on.

> *Expect this run to be long and to read as tedious. That is the correct shape for it: 44 items, each needing a status and evidence. A short, fluent audit that reads well is the failure mode to watch for, not the success case.*

```
You are performing an independent compliance audit. You have been
commissioned specifically because you did not write this code and did not
write the standard it is being judged against.

MATERIALS
- The complete engine source.
- The Phase-7 Engineering Constitution: a ratified 44-rule register
  (21 Tier 1 invariants, 7 Tier 2, 10 Tier 3, 6 Tier 4).
- A Step 2a audit package: a file manifest, evidence pointers, and a
  self-documented failure history.
- Runtime logs and one simulated trade record.

DISCLOSURE YOU SHOULD ACT ON
The audit package was assembled by Claude, which also wrote most of the
engine and co-drafted the Constitution. It is therefore not a neutral
document. Treat every factual claim in it - including its stated search
results - as a claim to be verified against the source yourself, not as
an established fact. Where you cannot verify one, say so.

YOUR TASK
Evaluate the engine against the register. Begin with the Minimum Viable
Audit gate - Items 2, 3, 6 and 18 - then work through the remaining Tier
1 invariants, then Tiers 2 to 4.

Record every finding in this schema:

  Principle       - which item, by number and name
  Status          - Compliant / Non-compliant / Unknown
  Severity        - Critical / Major / Moderate / Minor
  Effort          - rough cost to remedy
  Evidence        - the specific file, function or log line. Evidence
                    must be independently checkable by someone who has
                    not read your reasoning.
  Impact          - what goes wrong in practice if this stands
  Required action - what would have to change
  Verification    - how one would confirm the fix worked
  Re-audit        - whether this needs rechecking later

RULES THAT ARE NOT NEGOTIABLE
- "Unknown" is a legitimate and expected result. If a claim cannot be
  evidenced from the artefacts in front of you, the status is Unknown.
  Do not argue an Unknown into a Compliant because the code looks like it
  probably does the right thing.
- Do not treat the Constitution's own confidence about the engine as
  evidence about the engine.
- Two items are handed to you as open questions in the package's Section
  5 rather than as findings. Test them; do not confirm them.
- The Constitution's Version History contains a row labelled DEFECT,
  recording an internal contradiction about the composition of the
  Minimum Viable Audit gate. You are required to record a formal finding
  resolving which passage stands.
- Where your own reasoning may be correlated with that of the model that
  built this - shared training data, shared habits - say so explicitly at
  that point in the audit. Do not present a possibly-shared blind spot as
  an independent confirmation.

Do not open with an assessment of the document's quality. Start with
findings.
```

4. Contamination Rules

Each of these has a specific failure behind it, either in this project's own history or in the reasoning the Constitution was written to enforce.

Do not run Run 2 before Run 1.

The blind review is available once. See the note on the first page.

Do not use a model that has already discussed this project.

Grok reviewed the Constitution favourably on August 26 and was removed from the auditor plan partly for that reason — see Engineering Notes Entries #18 and #20. Gemini, ChatGPT and Copilot reviewed the document during drafting; Claude wrote it. All five are disqualified.

Do not tell the auditor what you are hoping it will find.

This includes framing like “I think Item 17 is fine, but check it” and equally “I suspect the backtesting isolation is broken.” Both convert a test into a confirmation.

Do not let an Unknown be argued into a Compliant.

Instruction 2 forbids it, but watch for it happening anyway — it usually appears as reasoning about what the code probably does rather than evidence of what it does. Under Item 8, Unknown is a legitimate result and does not need rescuing.

Do not treat agreement between auditors as validation.

Revision 3 already corrected this document for making exactly that mistake about its own reviewers. If Run 1 and Run 2 agree, that is weak evidence. Where they *disagree* is the part worth reading twice.

Do not paste Claude's answers back to the auditor.

Step 4a is Claude answering findings adversarially. Those answers go to Viktor, who adjudicates. Feeding them back to the auditor mid-audit turns an independent review into a negotiation.

5. After Both Runs

Bring both results back to Claude in full — not summarised, and including anything that reflects badly on Claude's own work, which is the part most at risk of being trimmed on the way. What follows is defined by the Constitution, not by this document:

Step 4a — Claude answers every finding adversarially.

Including, and especially, findings against code Claude wrote. The standard is not whether a finding is convenient but whether it is correct.

Viktor adjudicates disagreements.

Where Claude and an auditor disagree, neither resolves it internally. That decision is Viktor's, per Roles & Authority.

The DEFECT row must receive a formal finding.

The Minimum Viable Audit gate contradiction was recorded rather than repaired precisely so the auditor would resolve it. If Run 2 does not address it, ask again before closing the audit.

The scope freeze lifts only when every Tier 1 item has a recorded finding.

Compliant, Non-compliant or Unknown — all three count. Fixes do not have to have landed. An item with no finding at all leaves the freeze in force.

Then, and only then, decide whether more runs are worth it.

DeepSeek and GLM (Z.ai) remain available as further independent runs. Whether they add anything is a question the first two results answer. Adding auditors is easy and feels productive; answering findings honestly is the harder work and the part that improves the engine.

> *Both instructions above, and the method behind them, get recorded in Engineering Notes once the audit has run — alongside the findings they produced, rather than in advance of them. An instruction written down before any result exists cannot say the one thing worth knowing about it: whether it worked.*

6. Amendment — Instruction 2 Split Into Three Runs

Added August 27, 2026, after the first attempt at Run 2

Why this section exists.

Section 2 above states that the exact wording given to the auditor is preserved here *rather than paraphrased*, because under Item 6 how the auditor was asked bears on what the auditor found. Instruction 2 was then changed before it was ever used. Recording the change is the whole point of having written the original down; leaving it unrecorded would have made Section 3 a description of an instruction nobody ran.

What happened

The first attempt at Run 2 was sent with Instruction 2 exactly as recorded in Section 3, to Kimi K3 with all five materials attached. It produced a 16,384-token reasoning trace and then stopped, having written no findings at all. The cause was not the model: OpenRouter's chat interface leaves Max Tokens unset by default and falls back to 16,384, and reasoning tokens are billed against that same ceiling. The auditor spent its entire output budget thinking and was cut off before the answer began.

The trace was not wasted — it was read, and three of its claims were checked against the source and confirmed. But it contained no findings in the required schema, so it forms no part of the audit record. Settings were corrected (Max Tokens 64,000; reasoning enabled at maximum effort; OpenRouter's default system prompt cleared) and the audit was split.

The split

Maximum reasoning effort across all 44 items risked the same truncation at a higher ceiling. Rather than lower the effort, the register was divided across three runs, each getting the full budget for a quarter of the work. The division follows the Constitution's own priority order rather than being invented for convenience: the Minimum Viable Audit gate first, the remaining Tier 1 invariants second, Tiers 2 to 4 third.

Everything in Instruction 2 stays byte-identical across all three runs except the YOUR TASK paragraph, reproduced below in each of its three forms, plus one addition to the Evidence field noted after them. The DEFECT clause appears in Run A only, and is dropped from B and C once Run A has resolved it.

Run A — Minimum Viable Audit gate

```
This run covers the Minimum Viable Audit gate ONLY: Item 2 (Look-Ahead
Bias), Item 3 (Data Integrity), Item 6 (Traceability) and Item 18
(Read-Only Market Access), plus the DEFECT row resolution described
below. Do not proceed to the remaining Tier 1 invariants or to Tiers 2
to 4 - those are separate runs. Spend the depth you would have spent
across 44 items on these four instead.
```

Run B — remaining Tier 1 invariants

```
This run covers the Tier 1 invariants OUTSIDE the Minimum Viable Audit
gate: Items 1, 4, 5, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 19, 20 and
21. Items 2, 3, 6 and 18 were audited in a separate run and are out of
scope here - do not re-audit them. Do not proceed to Tiers 2 to 4; those
are a separate run.
```

Run C — Tiers 2 to 4

```
This run covers Tiers 2, 3 and 4 only - the 7 architectural principles,
the 10 process disciplines and the 6 stated preferences. All 21 Tier 1
invariants were audited in earlier runs and are out of scope here.
```

One addition to the Evidence field, from Run B onward

A withdrawn accusation, kept here rather than deleted.

This section originally justified the addition below with a claim that Run A's auditor had invented a citation — that it reported `[ERROR] Insufficient data` entries firing on the 1h and 1w scans which existed nowhere. That was wrong. The entries appear fourteen times in `scan_summary_report.txt`, one of the two evidence files supplied, each directly beneath an `ASSET / TIMEFRAME` header reading 1h or 1w. The auditor's claim was correct in every particular, including the 1h/1w detail; it named the wrong file, and its own reasoning trace names the right one. Claude had searched `phase7_engine.log` alone and concluded from that one search that the string did not exist — the same error, made hours apart, that produced the Layer 5 falsehood in the Step 2a package. The accusation is withdrawn.

The addition originally drafted told the auditor not to infer log entries from format strings in the source. That was aimed at a fabrication that never happened, and it carried a cost: an auditor warned off citing logs cites fewer of them, and Run A's log work was among its most useful. The error that did occur was one of attribution — two evidence files were supplied and the wrong one was named — and the checker was misled because he searched only the file the auditor named. So the addition from Run B onward is narrower, and asks for the thing that would have prevented the whole episode:

```
When you cite a log line, name which of the supplied files it is in.
```

Run 1's unverified citation — a specific issue in a third-party library's tracker — is a separate matter and remains unverified rather than disproved. One unverified citation is not a pattern, and no instruction is added on its account.

The DISCLOSURE paragraph also gains one sentence from Run B onward, since by then it is a fact rather than a caution: *At least one factual claim in that package has already been shown false by an earlier run; assume there may be others.* That refers to Section 5.2 of the Step 2a package, which stated that Roadmap Layer 5 did not appear anywhere in the nineteen files under that name. It appears three times. Run A found it.

What this amendment does not do.

It changes no rule, resolves no finding, and does not alter the register, which stays frozen at 21 / 7 / 10 / 6. Splitting an audit across three sessions is a scheduling decision forced by a token ceiling, not a judgement about scope. Every one of the 44 items still requires a recorded finding, and the freeze still lifts only when all 21 Tier 1 items have one.